TP1: the `getcnt` system call on xv6-RISCV

Pedro Pires, Sistemas Operacionais, UFMG

1. Overview

The kernel now counts how many times each system call has been made. A new system call, `int getcnt(int)`, returns the count for a given system call number, and a user program called `getcnt` lets you read it from the shell. All the diffs shown below come from `git diff efc8e2b`, which is the unmodified version of this fork.

There is one difference from the handout. **Here `getcnt` is system call 23, not 22.** This tree is not the original xv6: it uses `pause` instead of `sleep` and adds a `sync` system call, which already uses number 22. So 23 is the first free number.

File	Change	Section
`kernel/syscall.c`	The counter array and its lock, the increment, `sys_getcnt` and `syscallinit()`	2, 3, 5
`kernel/defs.h`, `kernel/main.c`	Declare `syscallinit()` and call it at boot	2
`kernel/syscall.h`	`#define SYS_getcnt 23`	4
`user/user.h`, `user/usys.pl`	The declaration and the generated `ecall` stub	4
`user/getcnt.c`, `Makefile`	The user program and its `UPROGS` entry	4

In total: 38 new lines and 1 changed line in 7 files, plus a 22-line user program.

2. Diff 1: the data structure that holds the counts

```
diff --git a/kernel/syscall.c b/kernel/syscall.c
index 1800f20..cb44c7a 100644
--- a/kernel/syscall.c
+++ b/kernel/syscall.c
@@ -130,9 +131,35 @@ static uint64 (*syscalls[])(void) = {
   [SYS_mkdir]   = sys_mkdir,
   [SYS_close]   = sys_close,
   [SYS_sync]    = sys_sync,
+  [SYS_getcnt]  = sys_getcnt,
   // clang-format on
 };

+static int syscall_count[NELEM(syscalls)]; // calls made, per syscall number
+static struct spinlock count_lock; // syscall_count++ is not atomic
+
+void syscallinit(void) {
+  initlock(&count_lock, "syscall_count"); // named, so panic() can report it
+}
```

```
diff --git a/kernel/defs.h b/kernel/defs.h
index 0ae6896..db88a68 100644
--- a/kernel/defs.h
+++ b/kernel/defs.h
@@ -137,6 +137,7 @@ void            argaddr(int, uint64 *);
 int             fetchstr(uint64, char*, int);
 int             fetchaddr(uint64, uint64*);
 void            syscall();
+void            syscallinit(void);
```

```
 // trap.c
 extern uint     ticks;
diff --git a/kernel/main.c b/kernel/main.c
index df58e32..b2a4c63 100644
--- a/kernel/main.c
+++ b/kernel/main.c
@@ -20,6 +20,7 @@ main()
     kvminit();          // create kernel page table
     kvminithart();      // turn on paging
     procinit();         // process table
+    syscallinit();      // system call counters
     trapinit();         // trap vectors
     trapinithart();     // install kernel trap vector
     plicinit();         // set up interrupt controller
```

Part of the second hunk of `kernel/syscall.c`*, plus* `kernel/defs.h` *and* `kernel/main.c`

- **Why an array indexed by the call number.** System call numbers start at 1 and have no gaps, so the number itself works as the array index. Reading or updating a counter is then just one memory access, with nothing to allocate. That matters, because this code runs on every single system call.
- **Why** `NELEM(syscalls)` **and not a fixed number.** `syscalls[]` is the dispatch table right above it, so the counter array always has exactly one slot for each valid number. If someone adds a system call later, they only have to edit one table instead of two.
- **Why the counts are global and not per process.** The example in the handout shows `./getcnt 22` answering 2, then 3, then 4. Each of those runs is a different process, because the shell creates a new one every time. If the counter lived inside `struct proc`, it would start over at each `fork` and the answer would always be 1.
- **Why there is a lock.** xv6 runs on three CPUs, and `syscall_count[num]++` is not a single instruction: it is a load, an add and a store. If two CPUs run those steps at the same time, both read the same old value and both write old+1, so one increment disappears. Section 6.3 shows this actually happening.
- **Why** `syscallinit()` **exists.** A static spinlock is already filled with zeros, and zero means unlocked, so it would almost work without it. But `initlock()` also stores the name of the lock, and xv6 reads that name when it panics. Without the name, a future locking bug would crash inside the panic handler instead of printing a useful message.

3. Diff 2: how the counter is updated

```
diff --git a/kernel/syscall.c b/kernel/syscall.c
index 1800f20..cb44c7a 100644
--- a/kernel/syscall.c
+++ b/kernel/syscall.c
@@ -140,7 +167,11 @@ syscall(void)
   struct proc *p = myproc();

   num = p->trapframe->a7;
-  if (num > 0 && num < NELEM(syscalls) && syscalls[num]) {
+  if (validsyscall(num)) {
+    acquire(&count_lock);
+    syscall_count[num]++; // before dispatch, so getcnt counts itself
+    release(&count_lock);
+
     // Use num to lookup the system call function for num, call it,
     // and store its return value in p->trapframe->a0
     p->trapframe->a0 = syscalls[num]();
```

Third hunk of `git diff efc8e2b -- kernel/syscall.c`

- **Why here.** Every system call enters the kernel through `syscall()`, which reads the call number from register `a7` and calls the right handler. Counting in this one place covers all 23 system calls without changing any `sys_*` function, and any call added in the future gets counted too.

- **Why before the call and not after.** This is what makes `getcnt` count itself, which is what the handout expects. Running `./getcnt 1` is already the first call to number 23, so the next `./getcnt 23` answers 2, then 3, then 4. Counting after the call would also never count `exit`, because it never comes back.
- **Why inside the check.** The value in `num` comes straight from register `a7`, and a user program can put anything there. If we counted before checking, a program could make the kernel write outside the array, at a position it chooses. That would let it corrupt kernel memory.
- **Why the lock is released before the handler runs.** This is not only about speed. `acquire()` turns off interrupts, and several handlers go to sleep, such as `read` on a pipe, `wait` and `pause`. Going to sleep while holding a spinlock deadlocks xv6.

4. Diff 3: from the user program into the kernel

```
diff --git a/Makefile b/Makefile
index af1e7a5..8654421 100644
--- a/Makefile
+++ b/Makefile
@@ -150,6 +150,7 @@ UPROGS=\
     $U/_forphan\
     $U/_dorphan\
     $U/_sync\
+    $U/_getcnt\

 fs.img: mkfs/mkfs README $(UPROGS)
     mkfs/mkfs fs.img README $(UPROGS)
diff --git a/kernel/syscall.h b/kernel/syscall.h
index 5a4a39f..cf36412 100644
--- a/kernel/syscall.h
+++ b/kernel/syscall.h
@@ -21,3 +21,4 @@
 #define SYS_mkdir  20
 #define SYS_close  21
 #define SYS_sync   22
+#define SYS_getcnt 23
diff --git a/user/user.h b/user/user.h
index 29aad59..27c4abf 100644
--- a/user/user.h
+++ b/user/user.h
@@ -25,6 +25,7 @@ char *sys_sbrk(int, int);
 int pause(int);
 int uptime(void);
 int sync(void);
+int getcnt(int);

 // ulib.c
 int stat(const char *, struct stat *);
diff --git a/user/usys.pl b/user/usys.pl
index 4d054e8..0dfc583 100755
--- a/user/usys.pl
+++ b/user/usys.pl
@@ -43,3 +43,4 @@ entry("sbrk");
 entry("pause");
 entry("uptime");
 entry("sync");
+entry("getcnt");
```

`git diff efc8e2b -- kernel/syscall.h user/user.h user/usys.pl Makefile`

`syscall.h` is included by both sides, so the number 23 is written in only one place and the kernel and the user program can never disagree about it. `user.h` gives C code the declaration `int getcnt(int)` that the handout asks for.

`usys.pl` is where user code actually reaches the kernel. It is a Perl script that runs during the build and writes `user/usys.S`, with one small assembly stub for each system call. Adding `entry("getcnt")`

produces this:

```
getcnt:
 li a7, SYS_getcnt
 ecall
 ret
```

So `getcnt()` is not a normal library function, it is three instructions. The RISC-V calling convention already put the argument in register `a0`. The stub puts the system call number in `a7` and runs `ecall`, which switches the CPU to supervisor mode and jumps to the kernel trap handler. When the kernel returns, the result is already in `a0`, which is where C expects a return value, so a simple `ret` is enough.

The `UPROGS` line is also needed. That list is what `mkfs` copies into `fs.img`. Without it the program still compiles, but the shell cannot find it.

```
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"

int main(int argc, char *argv[]) {
  int num, cnt;

  if (argc != 2) {
    fprintf(2, "usage: getcnt syscall_number\n");
    exit(1);
  }

  num = atoi(argv[1]); // atoi("abc") is 0, which the kernel rejects below
  cnt = getcnt(num);
  if (cnt < 0) {
    fprintf(2, "getcnt: %s is not a valid syscall number\n", argv[1]);
    exit(1);
  }

  printf("syscall %d has been called %d times\n", num, cnt);
  exit(0);
}
```

`user/getcnt.c` *(new file)*

The message is exactly the one in the handout. Bad input is handled instead of printing something meaningless like `-1 times`. Both error messages go to file descriptor 2 and the program exits with 1. Text that is not a number needs no special case, because `atoi("abc")` returns 0 and 0 is not a valid call number.

5. Diff 4: reading the argument from the trapframe

```
diff --git a/kernel/syscall.c b/kernel/syscall.c
index 1800f20..cb44c7a 100644
--- a/kernel/syscall.c
+++ b/kernel/syscall.c
@@ -130,9 +131,35 @@ static uint64 (*syscalls[])(void) = {
+static int validsyscall(int num) { // shared by syscall() and sys_getcnt()
+  return num > 0 && num < NELEM(syscalls) && syscalls[num];
+}
+
+static uint64 sys_getcnt(void) { // int getcnt(int num)
+  int num;
+  int n;
+
+  argint(0, &num);
+  if (!validsyscall(num)) // num is user-controlled; never index unchecked
+    return -1;
+
+  acquire(&count_lock);
+  n = syscall_count[num];
+  release(&count_lock);
```

```
+  return n;
+}
```

Rest of the second hunk of `kernel/syscall.c`

The stub left the argument in register `a0` and then trapped. Before the kernel handler runs, `uservec` saves all the user registers into the **trapframe**, a page that belongs to the process and is reached through `p->trapframe`, declared in `proc.h`. So by the time `sys_getcnt` runs, the user's `a0` is just a field in a struct. `argint` is the function that reads it:

```
static uint64
argraw(int n)
{
  struct proc *p = myproc();
  switch (n) {
  case 0:
    return p->trapframe->a0;
  ...
```

`kernel/syscall.c`, *not modified*

This is why `copyin()` is not needed here. The trapframe is kernel memory, so reading an integer argument is just a struct access. Only pointer arguments need `copyin`, because a pointer refers to the user address space and has to be translated and checked against `p->sz` first. The way back is the same idea: `syscall()` writes the return value into `p->trapframe->a0`.

`argint` does no checking at all. It copies 64 bits chosen by the user program and cuts them down to an `int`. Using that value as an index right away would read outside the array in both directions: `getcnt(99)` reads past the end, and `getcnt(-1)` reads before the start, since a negative value stays negative. `validsyscall()` rejects both. It is the same function the dispatcher uses, so the two can never disagree about which numbers are valid.

The handler is `static` and lives in `syscall.c`, not in `sysfile.c` as the handout suggests. That way `syscall_count` and `count_lock` can stay `static` too, and only the code that owns them can touch them. Putting the handler in another file would force the array to become global, with no benefit, and the counters have nothing to do with files anyway.

6. How it was tested

All the runs below are on QEMU with the default 3 CPUs.

6.1 The example from the handout

```
$ getcnt 1
syscall 1 has been called 2 times
$ getcnt 23
syscall 23 has been called 2 times
$ getcnt 23
syscall 23 has been called 3 times
$ getcnt 23
syscall 23 has been called 4 times
```

Two things are being checked. First, the count grows by one even though each command is a different process, which a per-process counter could not do (Section 2). Second, the first answer for number 23 is 2 and not 1, because the earlier `getcnt 1` was itself the first call to number 23. That shows the counter is increased before the handler runs (Section 3).

6.2 Invalid input

```
$ getcnt 0
getcnt: 0 is not a valid syscall number
$ getcnt 99
```

```
getcnt: 99 is not a valid syscall number
$ getcnt -1
getcnt: -1 is not a valid syscall number
$ getcnt
usage: getcnt syscall_number
$ getcnt abc
getcnt: abc is not a valid syscall number
```

The important cases are 99 and -1. Without the check described in Section 5, each one would make the kernel read outside the array. None of these cases causes a panic, and the shell keeps working after all of them.

6.3 Counting correctly, and with several CPUs

Watching a number grow does not prove it is correct. To check that, a small extra program was written and run in xv6. It compares counters before and after doing a known number of system calls, instead of looking at absolute values, because the shell keeps making system calls at the same time. It tests four things:

- 5 calls to `getpid()` increase that counter by exactly 5.
- Two `getcnt` calls in a row differ by exactly 1.
- Invalid numbers return -1.
- Several child processes created with `fork`, each making many calls, increase the counter by exactly the total number of calls.

```
$ cnttest
test1 delta=5 expect=5 OK
test2 delta=1 expect=1 OK
test3 OK
test4 delta=40000 expect=40000 OK
```

A test like this proves nothing if the problem it looks for cannot happen in the first place. So the lock was removed on purpose and the last test was repeated with more processes and more calls:

Setup	Expected	Got	Result
No lock, 8 processes x 20000 calls	160000	159906	**FAIL**, 94 lost
With lock, 8 processes x 20000 calls	160000	160000	**OK**
With lock, 4 processes x 10000 calls	40000	40000	**OK**

So the problem is real in this kernel, and the spinlock is what fixes it. It is worth noting that with fewer processes the version without the lock still passed. A weaker test would have suggested the lock was not needed.

6.4 Checking that nothing else broke

This change adds a lock to the path of every system call, so the real risk is breaking the rest of the kernel, not `getcnt` itself. Running `usertests` inside xv6 prints `ALL TESTS PASSED`, with all 64 tests passing. The crash recovery tests `./test-xv6.py forphan` and `./test-xv6.py dorphan` also pass.

`./test-xv6.py log` fails, but not because of this change. The original commit `efc8e2b` was built in a separate `git worktree` and fails the same test in the same way. The real cause is the operating system used to run the tests: the script looks for the QEMU process with `ps --no-headers --ppid`, which only exists on Linux, not on macOS.